

WatchEdge Project - Final Report

1. Overview

This project involves the implementation of the network architecture for WatchEdge, a distributed IoT-based wildlife monitoring and detection system designed to operate efficiently across networking environments with constrained and intermittent connectivity. It is important to note that the overall WatchEdge concept and the underlying AI detection model were provided; our core contribution centers entirely on building the communication, persistence, and storage infrastructure bounding the system.

The implemented system features a three-tier architecture - Extreme-Edge, Edge, and Cloud - that decouples local AI inference from central data aggregation:

- **Extreme-Edge:** The individual AI-equipped camera nodes deployed in the field.
- **Edge:** The local aggregation gateways manage nearby clusters of cameras.
- **Cloud:** The centralized remote datacenter overseeing the entire fleet.

By pushing real-time object detection computing to the sensor level, coupled with edge caching and resilient MQTT bridging, our network architecture ensures persistent data processing and reliable image media storage. Furthermore, all system components across all three tiers are heavily containerized using Docker to simplify distribution, configuration, and long-term lifecycle management.

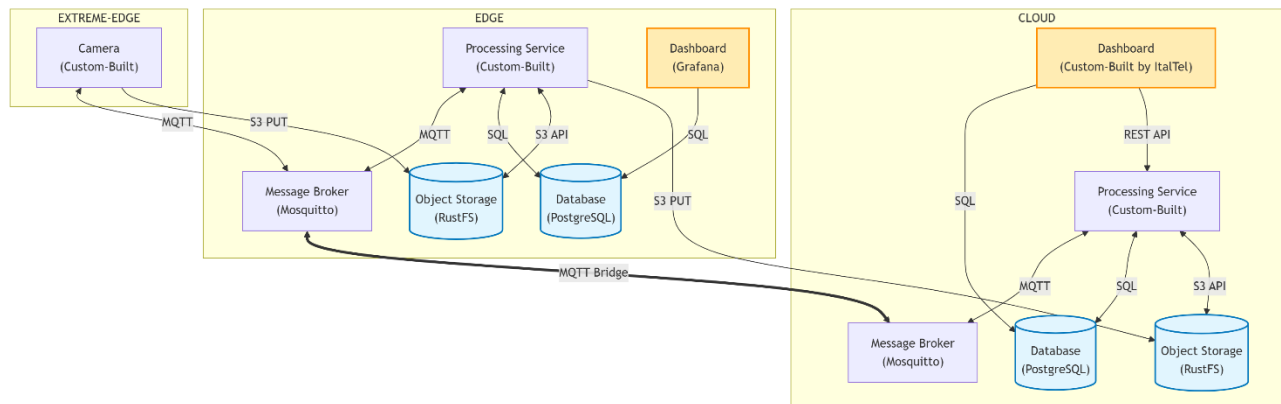


Figure 1 Global high-level system diagram.

2. Testing & Hardware Environment

While the proposed architecture is hardware-agnostic by design, the entire system has been deployed and validated on a specific hardware baseline. The hardware selected for this validation was:

- **Extreme-Edge: Nvidia Jetson Orin Nano.** Chosen to provide the necessary GPU acceleration required for real-time AI inference. It runs the custom YOLOv9t-based model fine-tuned specifically for the detection and classification of wild animals.
- **Edge: Raspberry Pi.** Selected for its optimal balance between low cost, low power consumption, and sufficient computational capacity to seamlessly host the local MQTT broker, lightweight database, and processing service.
- **Cloud (Central Infrastructure):** A single Virtual Private Server (VPS) hosted on **Scaleway**. Selected as a highly cost-effective yet reliable solution to operate the global broker, central database, and front-end management tools (such as the Custom ItalTel Dashboard) for validation purposes).

3. Extreme-Edge

The Extreme-Edge tier consists of physically deployed AI camera modules operating as the primary sensory nodes.



Figure 2 "Extreme-Edge" Lab Test: Live inference running on a Jetson Orin Nano from a video feed.

- **Camera Application:** This Python-based application manages video capture via OpenCV (supporting USB or pre-recorded video) and performs AI inference using a custom AI model fine-tuned via the YOLOv9 framework model. It also includes a localized management interface built with **FastAPI** and **Jinja2** to facilitate field administration, log monitoring, and live MJPEG streaming.

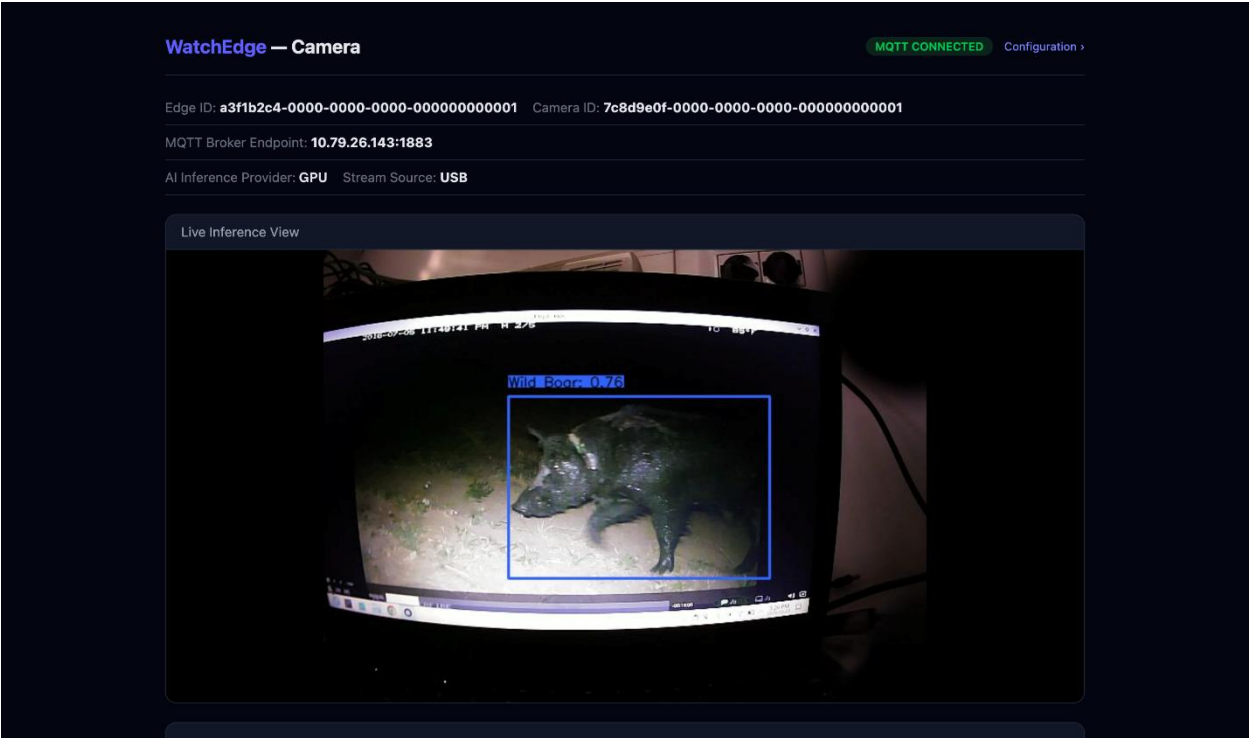


Figure 3 Boar identification.

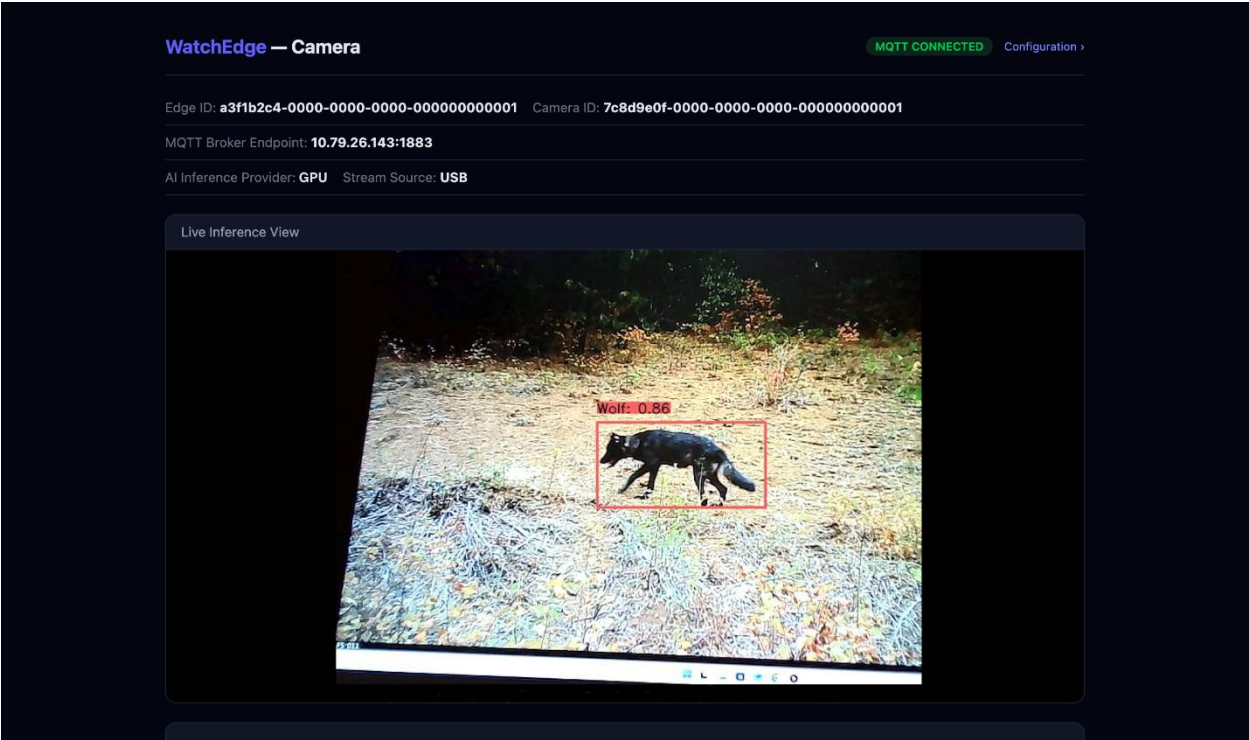


Figure 4 Wolf identification.

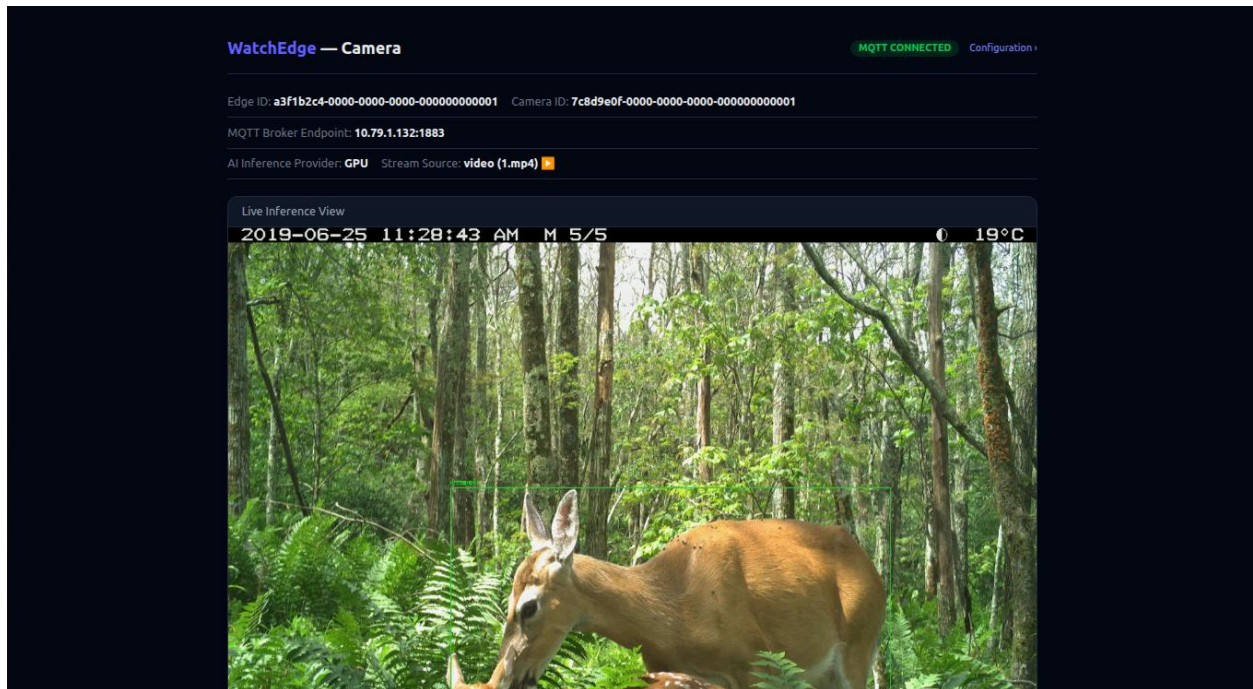


Figure 5 Deer identification (Video → OpenCV direct pipeline).

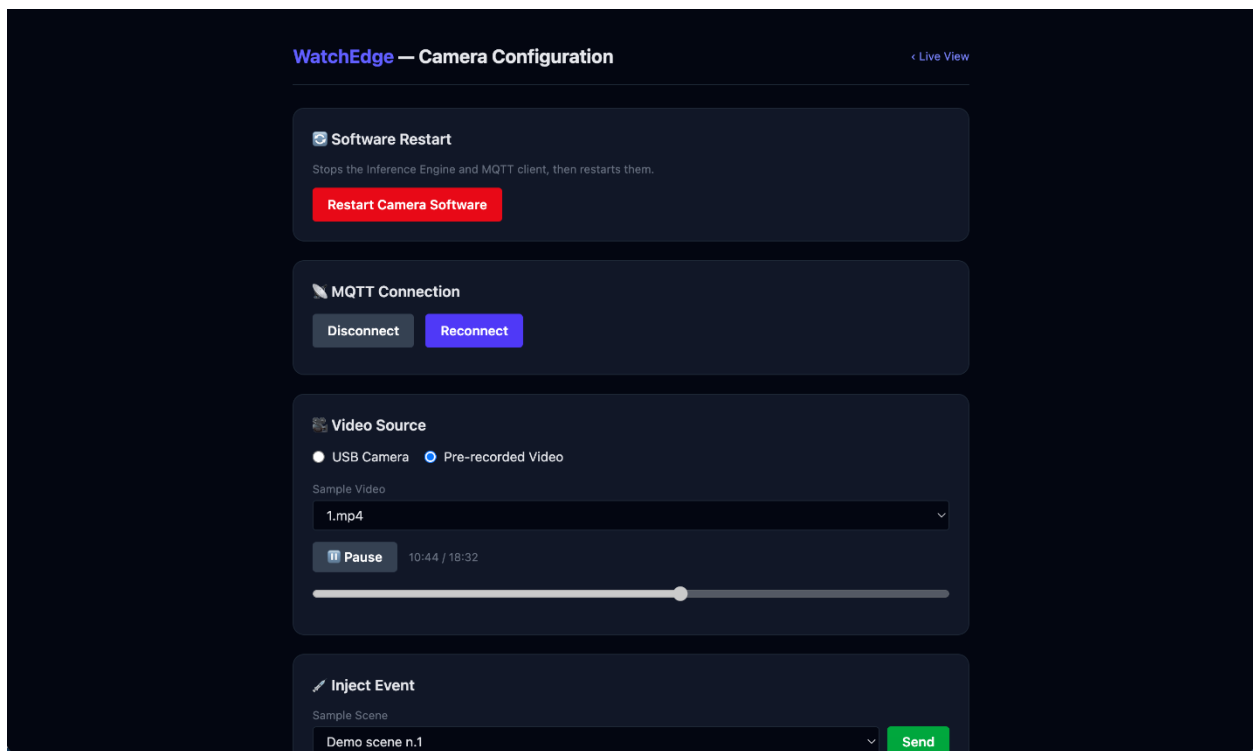


Figure 6 Configuration WebUI.

- **Telemetry and Event Pipeline:** Communication takes place over a Paho-MQTT client. Cameras publish lifecycle states, hardware telemetry (battery, coordinates), and animal detection schemas directly into the edge/ namespace. The system utilizes **QoS 1** for guaranteed event delivery alongside the **Last Will and Testament (LWT)** pattern to broadcast unexpected physical disconnects (e.g., sudden power loss).
- **Media Uploads:** Frames with positive detections are temporarily buffered. Following an event, the Edge Processing Service issues an MQTT command (cmd/upload) instructing the camera to offload the image to local Edge Object Storage via HTTP, ensuring the constrained local storage is rapidly freed.

4. Edge

The Edge tier sits between the remote cameras and the cloud network, acting as a local aggregation hub.



Figure 7 "Edge" Lab Test: Raspberry Pi serving as the local gateway node.

- **MQTT Broker (Mosquitto):** Functions as the local connective tissue. It utilizes **MQTT bridging** where the Edge broker initiates a single outbound connection to the Cloud broker. This provides "Store-and-Forward" capabilities, buffering messages locally during cloud disconnects and forwarding them once connectivity is restored.
- **Edge Processing Service:** A Python application (using **SQLModel** and **FastAPI**) that subscribes to local edge/# topics, validates telemetry, and caches metadata in a local **PostgreSQL** database. It orchestrates the image lifecycle by commanding cameras to upload detection frames to the local storage.

- **Local Persistence & Object Storage:** The tier utilizes **PostgreSQL with PostGIS** for geospatial metadata and **RustFS** for S3-compatible local object storage, ensuring cameras can offload images without an active internet connection.
- **Local Dashboard (Grafana):** A basic dashboard instance is deployed at the Edge for on-field support. It visualizes tabular data from the local DB and uses the Edge Processing Service's API as a proxy to retrieve inference images.

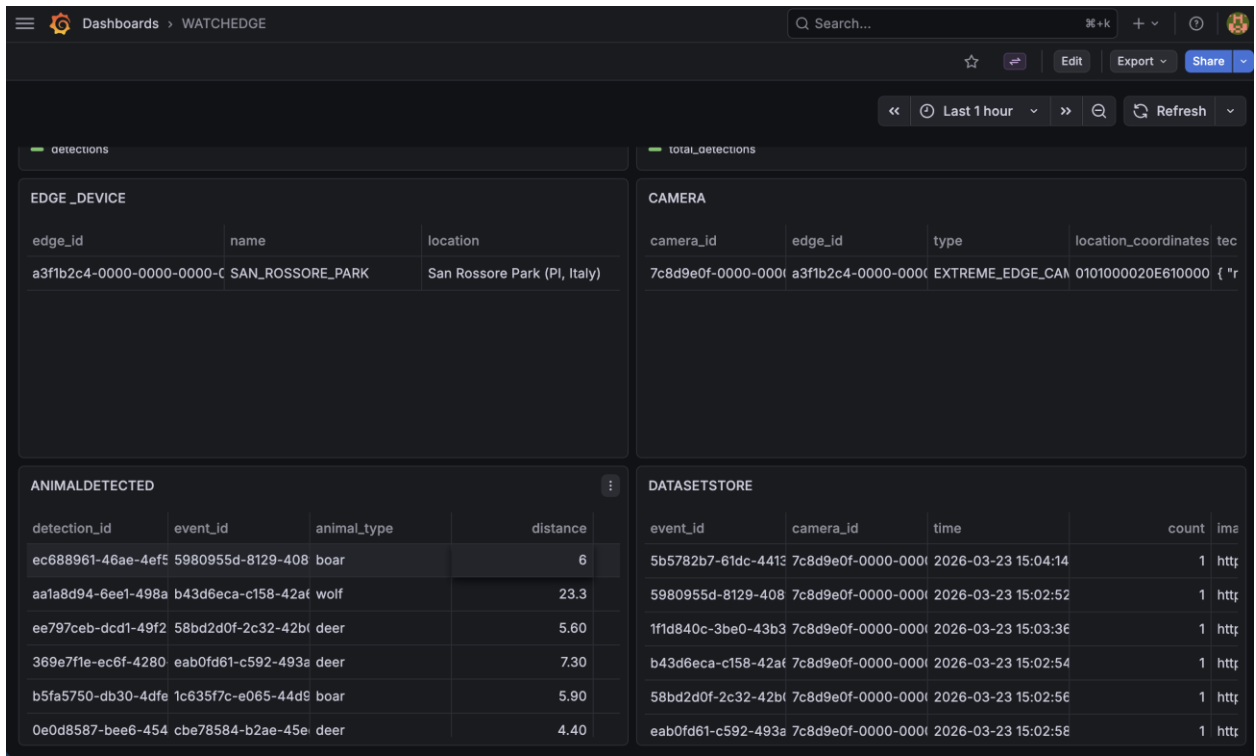


Figure 8 "Edge" Dashboard: Grafana interface monitoring the local gateway node.

5. Cloud

The Cloud tier acts as the master reporting and long-term storage layer for the WatchEdge fleet.

- **Central Data Persistence & MQTT Broker:** While the Edge is simple, the **Cloud Broker manages N simultaneous connections** (one per deployed Edge gateway). The traffic is compartmentalized using a **UUID-based namespace** (`ccloud/{edge_id}/{camera_id}/...`), ensuring no data collisions between different forest sectors and avoiding enumeration attacks.
- **Cloud Processing Service:** This is the same codebase as the Edge service but toggled into CLOUD mode via environment variables. It consumes bridged metrics and finalizes global database persistence. To save bandwidth, images are only

synchronized **on-demand**: if a user requests an image, the service sends a command down the MQTT bridge to the Edge, which then uploads the image to the Cloud RustFS bucket via HTTP.

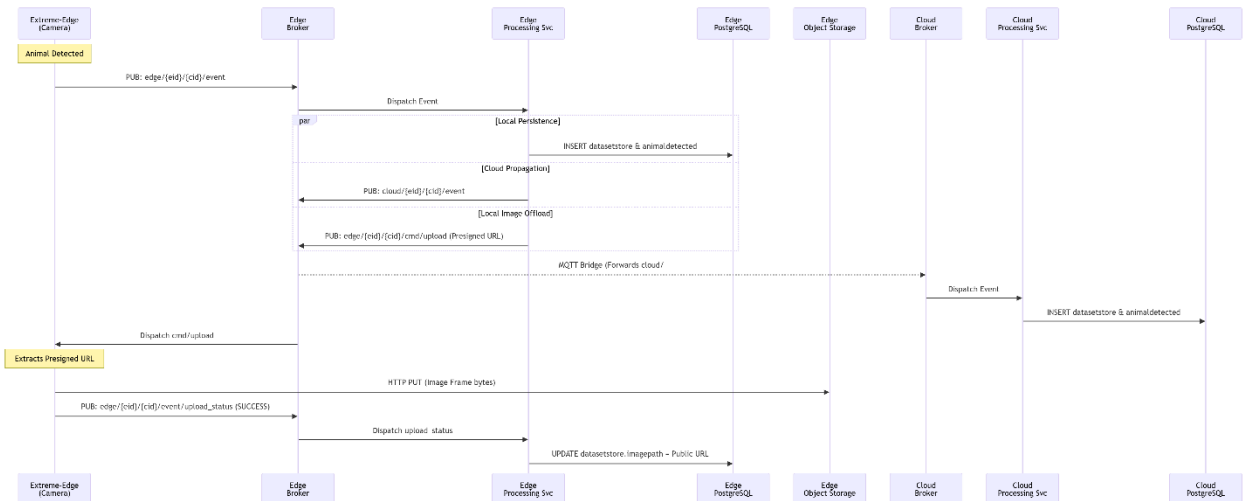


Figure 9 Event Ingestion Workflow.

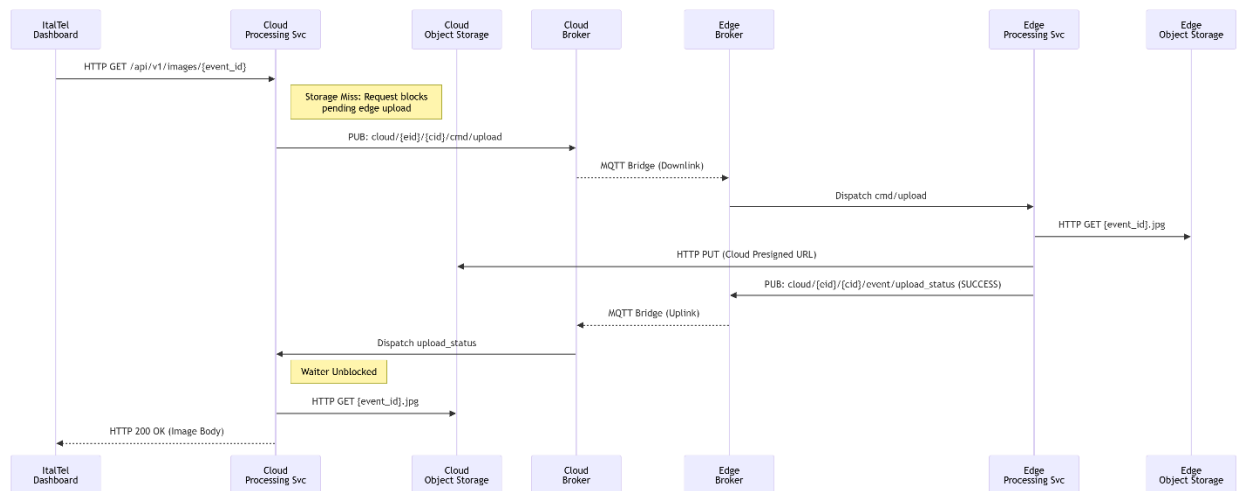


Figure 10 On-Demand Cloud Image Retrieval.

- **Dashboard Interface (ItaTel):** The platform integrates with a custom dashboard provided by ItaTel for advanced visualization. It pulls data from the Cloud database and uses the Cloud Processing Service's proxy to display images, allowing operators to monitor the fleet globally and evaluate AI inference quality.



Figure 11 ItalTel Dashboard Features Overview. A video demonstration of the interface in action is available.

6. Summary

The WatchEdge implementation uses a modular pattern to solve wildlife monitoring tasks via edge computing. The design philosophy separates the **control plane (MQTT)** from the **data plane (HTTP)**. By using UUID-based addressing, separate namespaces, and smart broker bridging, the system achieves a fault-tolerant infrastructure where Extreme-Edge nodes handle heavy CV tasks and Edge/Cloud services manage data synchronization and persistence seamlessly.

7. Future Work

Future development will focus on:

- **Power Management:** Dynamic throttling of inference framerates based on battery telemetry.
- **Zero-Touch Deployment:** Transitioning to TLS certificate-based authentication for automated provisioning.
- **High Availability (HA):** Implementing clustering for MQTT brokers and database replication to eliminate single points of failure.
- **Over-The-Air (OTA) Updates:** Implementing a remote update pipeline (e.g., Balena or K3s) for software and AI model upgrades.

8. Addendum: Database Schema

The database uses **PostgreSQL** with the **PostGIS** extension to support UUID-based relationships and geographic queries for camera locations.

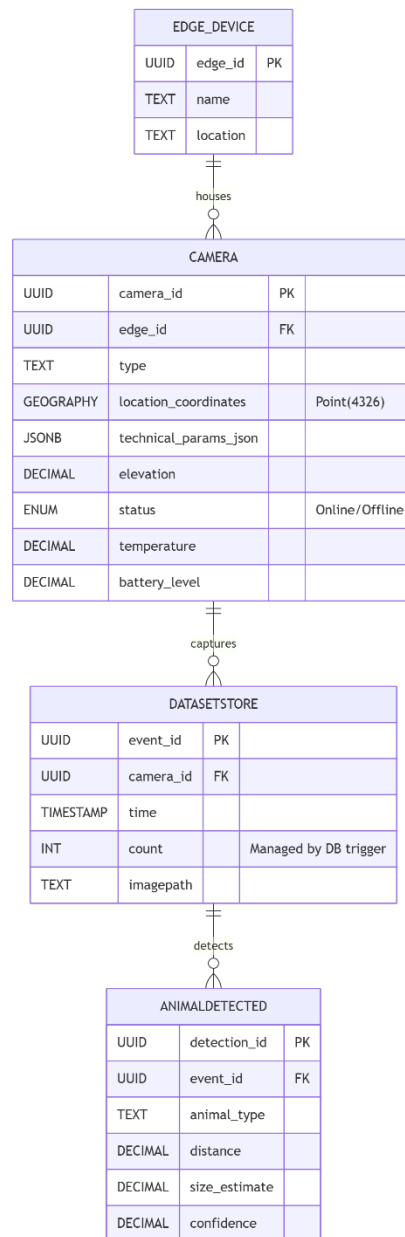


Figure 12 WatchEdge Database Schema.